

TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Analytics

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based ETL pipeline. It defines the technical architecture, data structures, transformation logic, and operational parameters required to build a production-grade solution that ingests customer and order data, applies data quality transformations, implements SCD Type 2 using Apache Hudi, and generates analytical aggregates.

Scope

This TRD provides technical specifications for:

- AWS Glue job configuration and runtime parameters
- Source and target data structures with schema definitions
- Data ingestion patterns from S3 CSV sources
- Data quality transformation logic (null removal, deduplication)
- AWS Glue Data Catalog integration patterns
- Apache Hudi SCD Type 2 implementation details
- Incremental processing logic for customer updates
- Aggregation logic for customer spend analytics
- Error handling, logging, and validation rules
- Partitioning and storage optimization strategies

This TRD does NOT cover:

- AWS infrastructure provisioning (VPC, subnets, security groups)
- IAM role and policy definitions
- AWS Glue job scheduling or orchestration configuration
- Cost optimization strategies
- Disaster recovery procedures
- Data visualization layer implementation

Assumptions

- AWS Glue version 3.0 or higher with Python 3.7+ and Spark 3.1+ is available

- Apache Hudi 0.10.0+ libraries are available in the Glue environment
- AWS Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` exists prior to job execution
- Source CSV files use comma (`,`) as delimiter and include header rows
- CSV files are encoded in UTF-8
- Date fields in source data are in ISO 8601 format (YYYY-MM-DD) or can be parsed by Spark's default date parser
- `CustId` is the primary key for customer data and foreign key in order data
- `OrderId` is the primary key for order data
- The string "Null" (case-sensitive) and actual NULL values are the only null representations requiring removal
- Duplicate detection is based on exact match across all fields
- For SCD Type 2, the business key for `ordersummary` is `CustId` + `OrderId`
- "Change" in SCD2 context means any modification to customer attributes (Name, EmailId, Region) or order attributes (ItemName, PricePerUnit, Qty, Date)
- Glue job has appropriate IAM permissions for S3 read/write and Glue Catalog operations
- S3 paths use standard AWS S3 URI format (s3://bucket/prefix/)
- Parquet format will be used for all curated and analytics tables unless otherwise specified
- Snappy compression will be used for Parquet files
- CloudWatch Logs retention is configured at the AWS account level
- No schema evolution is expected during initial implementation
- Data types not specified in FRD will be inferred by Spark during CSV parsing
- For Hudi tables, `OpTs` (operation timestamp) will use milliseconds since epoch
- For SCD2, `EndDate` for active records will be set to NULL
- Athena engine version 2 or higher will be used for querying cataloged tables
- No data encryption at rest requirements beyond S3 default encryption
- No data masking or PII redaction requirements
-

2. Technical Requirements

TR-INGEST-001: Ingest Customer Data from S3 CSV

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer master data from S3 CSV files into a Glue DynamicFrame for downstream processing.

- Source Details:

- Source Type: S3 CSV Files
- Source Path: ``s3://adif-sdlc/sdlc_wizard/customerdata/``
- Source Format: CSV with header
- Source Delimiter: Comma (`,`)
- Source Encoding: UTF-8
- Target Details:
- Target Type: In-memory Glue DynamicFrame
- Target Name: ``customer_raw_df``

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String (inferred)	N/A	Yes	
	Name	String (inferred)	N/A	Yes	
	EmailId	String (inferred)	N/A	Yes	
	Region	String (inferred)	N/A	Yes	

- Output Schema:

Same as Input Schema (in-memory DynamicFrame)

- Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession
2. Use ``glueContext.create_dynamic_frame.from_options()`` to read CSV files
3. Specify format options:
 - ``format="csv"``
 - ``connection_type="s3"``
 - ``format_options={"withHeader": True, "separator": ","}``
4. Load all CSV files matching the S3 path pattern
5. Infer schema from CSV header and data types from content
6. Return DynamicFrame for downstream processing

- Validation Rules:

- S3 path must exist and be accessible
- At least one CSV file must be present in the source path
- CSV files must contain header row with expected column names: CustId, Name, EmailId, Region
- CSV files must be parseable with comma delimiter

- Error Handling and Logging:

- If S3 path does not exist: Log error "Source path s3://adif-sdlc/sdlc_wizard/customerdata/ not found" and raise exception to fail job
- If no CSV files found: Log error "No CSV files found in source path" and raise exception to fail job
- If CSV parsing fails: Log error with file name and parsing exception details, raise exception to fail job
- If schema mismatch detected: Log error "Expected columns [CustId, Name, EmailId, Region] not found in CSV header" and raise exception to fail job
- Log info message: "Successfully loaded [N] records from customer CSV files"
- Runtime Parameters:
 - `--SOURCE\_CUSTOMER\_PATH`: S3 path for customer CSV files (default: `s3://adif-sdlc/sdlc\_wizard/customerdata/`)
 - `--ENABLE\_GLUE\_DATACATALOG`: Enable Glue Data Catalog integration (default: `true`)
- Operational Notes:
 - Use Glue DynamicFrame instead of Spark DataFrame to leverage Glue-specific optimizations
 - Enable Glue job bookmark if incremental processing is required in future iterations
 - Monitor CloudWatch Logs for ingestion metrics (record count, file count, processing time)

TR-INGEST-002: Ingest Order Data from S3 CSV

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order transaction data from S3 CSV files into a Glue DynamicFrame for downstream processing.

- Source Details:
 - Source Type: S3 CSV Files
 - Source Path: `s3://adif-sdlc/sdlc\_wizard/orderdata/`
 - Source Format: CSV with header
 - Source Delimiter: Comma (`,`)
 - Source Encoding: UTF-8
- Target Details:
 - Target Type: In-memory Glue DynamicFrame
 - Target Name: `order\_raw\_df`
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String (inferred)	N/A	Yes	
	ItemName	String (inferred)	N/A	Yes	
	PricePerUnit	Decimal (inferred)	N/A	Yes	
	Qty	Integer (inferred)	N/A	Yes	
	Date	String (inferred)	N/A	Yes	
	CustId	String (inferred)	N/A	Yes	

- Output Schema:

Same as Input Schema (in-memory DynamicFrame)

- Processing / Transformation Logic:

1. Initialize GlueContext and SparkSession
2. Use ``glueContext.create_dynamic_frame.from_options()`` to read CSV files
3. Specify format options:
 - ``format="csv"``
 - ``connection_type="s3"``
 - ``format_options={"withHeader": True, "separator": ","}``
4. Load all CSV files matching the S3 path pattern
5. Infer schema from CSV header and data types from content
6. Return DynamicFrame for downstream processing

- Validation Rules:

- S3 path must exist and be accessible
- At least one CSV file must be present in the source path
- CSV files must contain header row with expected column names: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- CSV files must be parseable with comma delimiter

- Error Handling and Logging:

- If S3 path does not exist: Log error "Source path s3://adif-sdlc/sdlc_wizard/orderdata/ not found" and raise exception to fail job
- If no CSV files found: Log error "No CSV files found in source path" and raise exception to fail job
- If CSV parsing fails: Log error with file name and parsing exception details, raise exception to fail job
- If schema mismatch detected: Log error "Expected columns [OrderId, ItemName, PricePerUnit, Qty, Date, CustId] not found in CSV header" and raise exception to fail job

- Log info message: "Successfully loaded [N] records from order CSV files"
- Runtime Parameters:
 - `--SOURCE\_ORDER\_PATH`: S3 path for order CSV files (default: `s3://adif-sdlc/sdlc\_wizard/orderdata/`)
 - `--ENABLE\_GLUE\_DATACATALOG`: Enable Glue Data Catalog integration (default: `true`)
- Operational Notes:
 - Use Glue DynamicFrame instead of Spark DataFrame to leverage Glue-specific optimizations
 - Enable Glue job bookmark if incremental processing is required in future iterations
 - Monitor CloudWatch Logs for ingestion metrics (record count, file count, processing time)
-

TR-CLEAN-001: Remove Null Values from Customer Dataset

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove all customer records containing the string "Null" or actual NULL values in any field to ensure data quality.

- Source Details:
 - Source Type: In-memory Glue DynamicFrame
 - Source Name: `customer\_raw\_df` (output from TR-INGEST-001)
- Target Details:
 - Target Type: In-memory Glue DynamicFrame
 - Target Name: `customer\_null\_cleaned\_df`
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	Yes	
	Name	String	N/A	Yes	
	EmailId	String	N/A	Yes	
	Region	String	N/A	Yes	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	No	

	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame for filtering operations
2. Apply filter to remove records where any field is NULL: ``df.filter(col("CustId").isNotNull() & col("Name").isNotNull() & col("EmailId").isNotNull() & col("Region").isNotNull())``
3. Apply filter to remove records where any field equals the string "Null" (case-sensitive): ``df.filter((col("CustId") != "Null") & (col("Name") != "Null") & (col("EmailId") != "Null") & (col("Region") != "Null"))``
4. Convert filtered DataFrame back to DynamicFrame
5. Return cleaned DynamicFrame

- Validation Rules:

- All fields in remaining records must be non-null
- All fields in remaining records must not equal the string "Null"
- Record count after cleaning must be ≥ 0

- Error Handling and Logging:

- If all records are removed: Log warning "All customer records removed due to null values. Record count: 0" but do not fail job
- Log info message: "Customer null cleaning complete. Records before: [N], Records after: [M], Records removed: [N-M]"
- If filtering operation fails: Log error with exception details and raise exception to fail job

- Runtime Parameters:

- ``--NULL_STRING_VALUE``: String value to treat as null (default: ``"Null"``)

- Operational Notes:

- Null removal is performed before deduplication to avoid unnecessary processing of invalid records
- Consider logging sample records that are removed for data quality auditing
- Monitor the percentage of records removed to detect data quality issues in source systems
-

TR-CLEAN-002: Remove Null Values from Order Dataset

- Related Functional Requirement ID(s): FR-CLEAN-002
- Objective:

Remove all order records containing the string "Null" or actual NULL values in any field to ensure data quality.

- Source Details:
- Source Type: In-memory Glue DynamicFrame
- Source Name: `order\_raw\_df` (output from TR-INGEST-002)

- Target Details:
- Target Type: In-memory Glue DynamicFrame
- Target Name: `order\_null\_cleaned\_df`

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	N/A	Yes	
	ItemName	String	N/A	Yes	
	PricePerUnit	Decimal	N/A	Yes	
	Qty	Integer	N/A	Yes	
	Date	String	N/A	Yes	
	CustId	String	N/A	Yes	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	
	CustId	String	N/A	No	

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame for filtering operations
2. Apply filter to remove records where any field is NULL: ``df.filter(col("OrderId").isNotNull() & col("ItemName").isNotNull() & col("PricePerUnit").isNotNull() & col("Qty").isNotNull() & col("Date").isNotNull() & col("CustId").isNotNull())``
3. Apply filter to remove records where any field equals the string "Null" (case-sensitive): ``df.filter((col("OrderId") != "Null") & (col("ItemName") != "Null") & (col("PricePerUnit") != "Null") & (col("Qty") != "Null") & (col("Date") != "Null") & (col("CustId") != "Null"))``
4. Convert filtered DataFrame back to DynamicFrame
5. Return cleaned DynamicFrame

- Validation Rules:
 - All fields in remaining records must be non-null
 - All fields in remaining records must not equal the string "Null"
 - Record count after cleaning must be ≥ 0
- Error Handling and Logging:
 - If all records are removed: Log warning "All order records removed due to null values. Record count: 0" but do not fail job
 - Log info message: "Order null cleaning complete. Records before: [N], Records after: [M], Records removed: [N-M]"
 - If filtering operation fails: Log error with exception details and raise exception to fail job
- Runtime Parameters:
 - `--NULL_STRING_VALUE``: String value to treat as null (default: ``Null``)
- Operational Notes:
 - Null removal is performed before deduplication to avoid unnecessary processing of invalid records
 - Consider logging sample records that are removed for data quality auditing
 - Monitor the percentage of records removed to detect data quality issues in source systems
-

TR-CLEAN-003: Remove Duplicate Customer Records

- Related Functional Requirement ID(s): FR-CLEAN-003
- Objective:

Identify and remove duplicate customer records based on exact match across all fields, retaining only unique records.

- Source Details:
 - Source Type: In-memory Glue DynamicFrame
 - Source Name: ``customer_null_cleaned_df`` (output from TR-CLEAN-001)
- Target Details:
 - Target Type: In-memory Glue DynamicFrame
 - Target Name: ``customer_deduplicated_df``
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	

	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	

- Output Schema:

Same as Input Schema (deduplicated records)

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame for deduplication operations
2. Apply `dropDuplicates()` method without specifying columns to remove exact duplicates across all fields
3. Convert deduplicated DataFrame back to DynamicFrame
4. Return deduplicated DynamicFrame

- Validation Rules:

- No two records in the output should have identical values across all fields
- Record count after deduplication must be $\leq$ record count before deduplication
- Record count after deduplication must be ≥ 0

- Error Handling and Logging:

- If all records are removed: Log warning "All customer records removed during deduplication. Record count: 0" but do not fail job
- Log info message: "Customer deduplication complete. Records before: [N], Records after: [M], Duplicates removed: [N-M]"
- If deduplication operation fails: Log error with exception details and raise exception to fail job

- Runtime Parameters:

None specific to this requirement

- Operational Notes:

- Deduplication is based on exact match across all fields, not just primary key
- If business logic requires keeping the most recent duplicate based on a timestamp, that logic is not implemented in this requirement
- Monitor the number of duplicates removed to detect data quality issues in source systems
- Consider implementing a separate audit log for removed duplicates if required for compliance
-

TR-CLEAN-004: Remove Duplicate Order Records

- Related Functional Requirement ID(s): FR-CLEAN-004

- Objective:

Identify and remove duplicate order records based on exact match across all fields, retaining only unique records.

- Source Details:

- Source Type: In-memory Glue DynamicFrame
- Source Name: `order\_null\_cleaned\_df` (output from TR-CLEAN-002)

- Target Details:

- Target Type: In-memory Glue DynamicFrame
- Target Name: `order\_deduplicated\_df`

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	
	CustId	String	N/A	No	

- Output Schema:

Same as Input Schema (deduplicated records)

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame for deduplication operations
2. Apply `dropDuplicates()` method without specifying columns to remove exact duplicates across all fields
3. Convert deduplicated DataFrame back to DynamicFrame
4. Return deduplicated DynamicFrame

- Validation Rules:

- No two records in the output should have identical values across all fields
- Record count after deduplication must be $\leq$ record count before deduplication
- Record count after deduplication must be ≥ 0

- Error Handling and Logging:

- If all records are removed: Log warning "All order records removed during deduplication. Record count: 0" but do not fail job

- Log info message: "Order deduplication complete. Records before: [N], Records after: [M], Duplicates removed: [N-M]"
- If deduplication operation fails: Log error with exception details and raise exception to fail job
- Runtime Parameters:

None specific to this requirement

- Operational Notes:
- Deduplication is based on exact match across all fields, not just primary key
- If business logic requires keeping the most recent duplicate based on a timestamp, that logic is not implemented in this requirement
- Monitor the number of duplicates removed to detect data quality issues in source systems
- Consider implementing a separate audit log for removed duplicates if required for compliance
-

TR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned and deduplicated customer data to S3 in Parquet format and register it as a queryable table in AWS Glue Data Catalog.

- Source Details:
- Source Type: In-memory Glue DynamicFrame
- Source Name: `customer\_deduplicated\_df` (output from TR-CLEAN-003)
- Target Details:
- Target Type: S3 Parquet Files + Glue Catalog Table
- Target S3 Path: `s3://adif-sdlc/curated/sdlc\_wizard/customer/`
- Target Database: `gen\_ai\_poc\_databrickscoe`
- Target Table: `sdlc\_wizard\_customer`
- Target Format: Parquet with Snappy compression
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	

	Region	String	N/A	No	
--	--------	--------	-----	----	--

- Output Schema:

Same as Input Schema (persisted in Parquet format)

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame if necessary
2. Use ``glueContext.write_dynamic_frame.from_catalog()`` or ``glueContext.getSink()`` to write data
3. Specify write options:

- ``connection_type="s3"``
- ``format="parquet"``
- ``compression="snappy"``
- ``path="s3://adif-sdlc/curated/sdlc_wizard/customer/"``

4. Register table in Glue Data Catalog:

- ``database="gen_ai_poc_databrickscoe"``
- ``table_name="sdlc_wizard_customer"``
- ``updateBehavior="UPDATE_IN_DATABASE"``

5. Execute write operation

6. Verify table registration in Glue Catalog

- Validation Rules:

- Target S3 path must be accessible for write operations
- Glue Data Catalog database ``gen_ai_poc_databrickscoe`` must exist
- Table metadata must accurately reflect the customer schema
- Parquet files must be valid and readable by Athena

- Error Handling and Logging:

- If database does not exist: Log error "Glue Data Catalog database 'gen_ai_poc_databrickscoe' not found" and raise exception to fail job
- If S3 write fails: Log error "Failed to write customer data to S3 path `s3://adif-sdlc/curated/sdlc_wizard/customer/`` with exception details and raise exception to fail job
- If catalog registration fails: Log error "Failed to register table 'sdlc_wizard_customer' in Glue Data Catalog" with exception details and raise exception to fail job
- Log info message: "Successfully wrote [N] customer records to S3 and registered table in Glue Catalog"

- Runtime Parameters:

- ``--TARGET_CUSTOMER_PATH``: S3 path for customer Parquet files (default: ``s3://adif-sdlc/curated/sdlc_wizard/customer/``)

- `--CATALOG_DATABASE``: Glue Data Catalog database name (default: ``gen_ai_poc_databricksco``)
- `--CATALOG_CUSTOMER_TABLE``: Glue Data Catalog table name for customer (default: ``sdlc_wizard_customer``)
- Operational Notes:
 - Use overwrite mode for initial load; consider append or upsert for incremental loads
 - Parquet format provides efficient columnar storage and compression
 - Snappy compression offers good balance between compression ratio and speed
 - Table is immediately queryable via Athena after successful registration
 - Consider partitioning by Region if query patterns support it (not specified in FRD)
-

TR-CATALOG-002: Register Order Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Write cleaned and deduplicated order data to S3 in Parquet format and register it as a queryable table in AWS Glue Data Catalog.

- Source Details:
 - Source Type: In-memory Glue DynamicFrame
 - Source Name: ``order_deduplicated_df`` (output from TR-CLEAN-004)
- Target Details:
 - Target Type: S3 Parquet Files + Glue Catalog Table
 - Target S3 Path: ``s3://adif-sdlc/curated/sdlc_wizard/order/``
 - Target Database: ``gen_ai_poc_databricksco``
 - Target Table: ``sdlc_wizard_order``
 - Target Format: Parquet with Snappy compression
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	

	CustId	String	N/A	No	
--	--------	--------	-----	----	--

- Output Schema:

Same as Input Schema (persisted in Parquet format)

- Processing / Transformation Logic:

1. Convert DynamicFrame to Spark DataFrame if necessary
2. Use ``glueContext.write_dynamic_frame.from_catalog()`` or ``glueContext.getSink()`` to write data
3. Specify write options:

- ``connection_type="s3"``
- ``format="parquet"``
- ``compression="snappy"``
- ``path="s3://adif-sdlc/curated/sdlc_wizard/order/"``

4. Register table in Glue Data Catalog:

- ``database="gen_ai_poc_databrickscoe"``
- ``table_name="sdlc_wizard_order"``
- ``updateBehavior="UPDATE_IN_DATABASE"``

5. Execute write operation

6. Verify table registration in Glue Catalog

- Validation Rules:

- Target S3 path must be accessible for write operations
- Glue Data Catalog database ``gen_ai_poc_databrickscoe`` must exist
- Table metadata must accurately reflect the order schema
- Parquet files must be valid and readable by Athena

- Error Handling and Logging:

- If database does not exist: Log error "Glue Data Catalog database 'gen_ai_poc_databrickscoe' not found" and raise exception to fail job
- If S3 write fails: Log error "Failed to write order data to S3 path `s3://adif-sdlc/curated/sdlc_wizard/order/`` with exception details and raise exception to fail job
- If catalog registration fails: Log error "Failed to register table 'sdlc_wizard_order' in Glue Data Catalog" with exception details and raise exception to fail job
- Log info message: "Successfully wrote [N] order records to S3 and registered table in Glue Catalog"

- Runtime Parameters:

- ``--TARGET_ORDER_PATH``: S3 path for order Parquet files (default: ``s3://adif-sdlc/curated/sdlc_wizard/order/``)

- `--CATALOG_DATABASE``: Glue Data Catalog database name (default: ``gen_ai_poc_databrickscoe``)
- `--CATALOG_ORDER_TABLE``: Glue Data Catalog table name for order (default: ``sdlc_wizard_order``)

- Operational Notes:

- Use overwrite mode for initial load; consider append or upsert for incremental loads
- Parquet format provides efficient columnar storage and compression
- Snappy compression offers good balance between compression ratio and speed
- Table is immediately queryable via Athena after successful registration
- Consider partitioning by Date if query patterns support it (not specified in FRD)

-

TR-SCD2-001: Implement SCD Type 2 for Order Summary Using Apache Hudi

- Related Functional Requirement ID(s): FR-SCD2-001

- Objective:

Join customer and order datasets, apply Slowly Changing Dimension Type 2 logic to track historical changes, and persist the result as a Hudi table in S3 with Glue Catalog registration.

- Source Details:

- Source Type 1: In-memory Glue DynamicFrame (Customer)
- Source Name 1: ``customer_deduplicated_df`` (output from TR-CLEAN-003)
- Source Type 2: In-memory Glue DynamicFrame (Order)
- Source Name 2: ``order_deduplicated_df`` (output from TR-CLEAN-004)
- Source Type 3: Existing Hudi Table (if present)
- Source Path 3: ``s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``

- Target Details:

- Target Type: S3 Hudi Table + Glue Catalog Table
- Target S3 Path: ``s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``
- Target Database: ``gen_ai_poc_databrickscoe``
- Target Table: ``ordersummary``
- Target Format: Apache Hudi (Copy-on-Write)

- Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	

	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	

• Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	
	CustId	String	N/A	No	

• Output Schema (Order Summary with SCD2 Fields):

Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
-----	-----	-----	-----	-----
CustId	String	N/A	No	Direct map from Customer.CustId and Order.CustId (join key)
Name	String	N/A	No	Direct map from Customer.Name
EmailId	String	N/A	No	Direct map from Customer.EmailId
Region	String	N/A	No	Direct map from Customer.Region
OrderId	String	N/A	No	Direct map from Order.OrderId
ItemName	String	N/A	No	Direct map from Order.ItemName
PricePerUnit	Decimal	N/A	No	Direct map from Order.PricePerUnit
Qty	Integer	N/A	No	Direct map from Order.Qty
Date	String	N/A	No	Direct map from Order.Date
IsActive	Boolean	N/A	No	Derived: True for current active record, False for historical records
StartDate	Timestamp	N/A	No	Derived: Timestamp when record became active
EndDate	Timestamp	N/A	Yes	Derived: Timestamp when record became inactive (NULL for active records)
OpTs	Long	N/A	No	Derived: Operation timestamp in milliseconds since epoch

• Processing / Transformation Logic:

1. Join Customer and Order Data:

- Convert customer and order DynamicFrames to Spark DataFrames
- Perform inner join on `CustId`: `joined\_df = customer\_df.join(order\_df, "CustId", "inner")`
- Result contains all customer and order fields

2. Read Existing Hudi Table (if exists):

- Check if Hudi table exists at `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`

- If exists, read using: ``existing_df = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")``
- If not exists, create empty DataFrame with target schema

3. Identify Changes:

- Define business key: ``CustId` + `OrderId``
- For each record in ``joined_df``, compare with existing active records in ``existing_df`` (where ``IsActive = True``)
- A change is detected if any of the following fields differ: Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date

4. Apply SCD Type 2 Logic:

- For new records (business key not in existing table):
 - Add SCD2 fields: ``IsActive = True``, ``StartDate = current_timestamp()``, ``EndDate = NULL``, ``OpTs = current_timestamp_millis()``
 - Insert into target dataset
- For changed records (business key exists but attributes differ):
 - Update existing active record: Set ``IsActive = False``, ``EndDate = current_timestamp()``, ``OpTs = current_timestamp_millis()``
 - Insert new active record: Set ``IsActive = True``, ``StartDate = current_timestamp()``, ``EndDate = NULL``, ``OpTs = current_timestamp_millis()``
- For unchanged records (business key exists and attributes match):
 - No action required; retain existing record as-is

5. Write to Hudi Table:

- Configure Hudi options:
 - ``hoodie.table.name = "ordersummary"``
 - ``hoodie.datasource.write.recordkey.field = "CustId,OrderId"``
 - ``hoodie.datasource.write.precombine.field = "OpTs"``
 - ``hoodie.datasource.write.partitionpath.field = ""`` (no partitioning unless specified)
 - ``hoodie.datasource.write.operation = "upsert"``
 - ``hoodie.datasource.write.table.type = "COPY_ON_WRITE"``
 - ``hoodie.datasource.hive_sync.enable = "true"``
 - ``hoodie.datasource.hive_sync.database = "gen_ai_poc_databrickscoe"``
 - ``hoodie.datasource.hive_sync.table = "ordersummary"``
 - ``hoodie.datasource.hive_sync.partition_extractor_class = "org.apache.hudi.hive.NonPartitionedExtractor"``

- Write DataFrame to Hudi: ``final_df.write.format("hudi").options(hudi_options).mode("append").save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")``

6. Verify SCD2 Integrity:

- After write, verify that only one active record exists per business key
- Verify that historical records have ``IsActive = False`` and non-null ``EndDate``
- Validation Rules:
 - Join must produce at least 0 records (empty result is valid if no matching CustId)
 - Business key (CustId + OrderId) must be unique for active records (``IsActive = True``)
 - For each business key, there must be exactly one record with ``IsActive = True``
 - Historical records must have ``IsActive = False`` and ``EndDate` <= current timestamp`
 - Active records must have ``IsActive = True`` and ``EndDate = NULL``
 - ``StartDate`` must be `<= `EndDate`` for historical records
 - ``OpTs`` must be populated for all records
- Error Handling and Logging:
 - If join produces no records: Log warning "Join between customer and order data produced 0 records" but complete the job
 - If Hudi write fails: Log error "Failed to write order summary to Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/" with exception details and raise exception to fail job
 - If multiple active records exist for the same business key after processing: Log error "Data integrity violation: Multiple active records found for business key [CustId, OrderId]" and raise exception to fail job
 - If Hive sync fails: Log error "Failed to sync Hudi table with Glue Data Catalog" with exception details and raise exception to fail job
 - Log info message: "SCD Type 2 processing complete. New records: [N], Changed records: [M], Unchanged records: [P]"
- Runtime Parameters:
 - ``--TARGET_ORDERSUMMARY_PATH``: S3 path for order summary Hudi table (default: ``s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``)
 - ``--CATALOG_DATABASE``: Glue Data Catalog database name (default: ``gen_ai_poc_databrickscoe``)
 - ``--CATALOG_ORDERSUMMARY_TABLE``: Glue Data Catalog table name for order summary (default: ``ordersummary``)
 - ``--HUDI_TABLE_TYPE``: Hudi table type (default: ``COPY_ON_WRITE``)
- Operational Notes:
 - Hudi Copy-on-Write (CoW) table type is recommended for read-heavy workloads

- Hudi Merge-on-Read (MoR) table type can be used for write-heavy workloads (not specified in FRD)
- Hudi automatically handles upsert logic based on record key and precombine field
- Hive sync enables immediate queryability via Athena
- Consider enabling Hudi clustering and compaction for large datasets
- Monitor Hudi commit timeline for performance optimization
- SCD Type 2 logic requires careful handling of timestamp precision to avoid race conditions
-

TR-INCR-001: Trigger SCD2 Updates on Customer Changes

- Related Functional Requirement ID(s): FR-INCR-001
- Objective:

Detect changes in customer data and trigger corresponding SCD Type 2 updates in the `ordersummary` Hudi table without full reprocessing.

- Source Details:
 - Source Type 1: In-memory Glue DynamicFrame (New/Updated Customer Data)
 - Source Name 1: `customer\_deduplicated\_df` (output from TR-CLEAN-003)
 - Source Type 2: Existing Hudi Table (Order Summary)
 - Source Path 2: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
- Target Details:
 - Target Type: S3 Hudi Table (Updated)
 - Target S3 Path: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
 - Target Database: `gen\_ai\_poc\_databrickscoe`
 - Target Table: `ordersummary`
- Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	

- Input Schema (Existing Order Summary):

Same as output schema from TR-SCD2-001

- Output Schema:

Same as output schema from TR-SCD2-001 (updated records)

- Processing / Transformation Logic:

1. Identify Changed Customers:

- Read existing `ordersummary` Hudi table
- Extract unique customer records from `ordersummary` where `IsActive = True`
- Compare with incoming customer data (`customer\_deduplicated\_df`)
- Identify customers where Name, EmailId, or Region has changed
- Identify new customers not present in `ordersummary`

2. Filter Affected Order Summary Records:

- For each changed or new customer, filter `ordersummary` records by `CustId`
- Result is a subset of `ordersummary` records that need SCD2 updates

3. Apply SCD Type 2 Logic:

- For each affected `CustId`:
- Update existing active records: Set `IsActive = False`, `EndDate = current\_timestamp()`, `OpTs = current\_timestamp\_millis()`
- Create new active records with updated customer attributes: Set `IsActive = True`, `StartDate = current\_timestamp()`, `EndDate = NULL`, `OpTs = current\_timestamp\_millis()`
- Retain order attributes (OrderId, ItemName, PricePerUnit, Qty, Date) unchanged

4. Write Updates to Hudi Table:

- Use Hudi upsert operation to write updated records
- Hudi configuration same as TR-SCD2-001
- Hudi will automatically handle record versioning based on `OpTs` precombine field

5. Verify Incremental Update:

- Verify that only affected records were updated
- Verify that unaffected records remain unchanged

• Validation Rules:

- Change detection must correctly identify new, changed, and unchanged customers
- Only records related to changed customers should be updated
- SCD2 integrity rules from TR-SCD2-001 must be maintained
- Incremental processing must produce the same result as full reprocessing for affected records

• Error Handling and Logging:

- If change detection fails: Log error "Failed to detect customer changes" with exception details and raise exception to fail job
- If no changes detected: Log info "No customer changes detected. Incremental processing skipped." and complete job successfully
- If Hudi upsert fails: Log error "Failed to apply incremental updates to order summary Hudi table" with exception details and raise exception to fail job
- Log info message: "Incremental processing complete. Customers changed: [N], Order summary records updated: [M]"
- Runtime Parameters:
 - `--ENABLE_INCREMENTAL_PROCESSING``: Enable incremental processing mode (default: `true``)
 - `--CHANGE_DETECTION_WATERMARK``: Timestamp watermark for change detection (optional)
- Operational Notes:
 - Incremental processing requires a mechanism to identify changed customers (e.g., timestamp comparison, change data capture)
 - If no change detection mechanism is available, full reprocessing (TR-SCD2-001) should be used
 - Consider using Glue job bookmarks to track processed data
 - Incremental processing reduces compute costs and processing time for large datasets
 - Monitor the ratio of incremental vs. full processing to optimize job scheduling

TR-AGG-001: Generate Customer Aggregate Spend Table

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Aggregate order data by customer to calculate total spend and persist the result as an analytics table in S3 with Glue Catalog registration.

- Source Details:
 - Source Type: Existing Hudi Table (Order Summary)
 - Source Path: `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``
- Target Details:
 - Target Type: S3 Parquet Files + Glue Catalog Table
 - Target S3 Path: `s3://adif-sdlc/analytics/customeraggregatespend/``
 - Target Database: `gen_ai_poc_databrickscoe``
 - Target Table: `customeraggregatespend``

- Target Format: Parquet with Snappy compression

- Input Schema (Order Summary - Active Records Only):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	
	IsActive	Boolean	N/A	No	
	StartDate	Timestamp	N/A	No	
	EndDate	Timestamp	N/A	Yes	
	OpTs	Long	N/A	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	Name	String	N/A	No	Direct map from OrderSummary.Name	
	Date	String	N/A	No	Direct map from OrderSummary.Date	
	TotalAmount	Decimal	N/A	No	Derived: SUM(PricePerUnit * Qty) grouped by Name and Date	

- Processing / Transformation Logic:

1. Read Order Summary Hudi Table:

- Read from ``s3://adif-sdlc/curated/sdlc_wizard/ordersummary/``

- Use Hudi read format: ``spark.read.format("hudi").load()``

2. Filter Active Records:

- Apply filter: ``df.filter(col("IsActive") == True)``

- Result contains only current active order summary records

3. Calculate Total Amount:

- Add calculated column: ``df.withColumn("TotalAmount", col("PricePerUnit") col("Qty"))``

4. Aggregate by Customer and Date:

- Group by: ``Name`, `Date``

- Aggregate: `SUM(TotalAmount)` as `TotalAmount`
- SQL equivalent: `SELECT Name, Date, SUM(PricePerUnit Qty) AS TotalAmount FROM ordersummary WHERE IsActive = True GROUP BY Name, Date`

5. Write to S3 and Register in Glue Catalog:

- Write aggregated DataFrame to S3 in Parquet format with Snappy compression
- Target path: `s3://adif-sdlc/analytics/customeraggregatespend/`
- Register table in Glue Data Catalog:
- Database: `gen_ai_poc_databrickscoe`
- Table: `customeraggregatespend`
- Update behavior: Overwrite
- Validation Rules:
- Only active records (`IsActive = True`) should be included in aggregation
- `TotalAmount` must be calculated as `PricePerUnit Qty` for each order
- Aggregation must group by both `Name` and `Date`
- Output must contain at least 0 records (empty result is valid if no active records exist)
- `TotalAmount` must be non-negative (assuming PricePerUnit and Qty are non-negative)
- Error Handling and Logging:
- If order summary table does not exist: Log error "Order summary Hudi table not found at `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`" and raise exception to fail job
- If no active records exist: Log warning "No active records found in order summary. Creating empty customer aggregate spend table." and create empty table
- If aggregation fails: Log error "Failed to aggregate customer spend" with exception details and raise exception to fail job
- If S3 write fails: Log error "Failed to write customer aggregate spend to S3 path `s3://adif-sdlc/analytics/customeraggregatespend/`" with exception details and raise exception to fail job
- If catalog registration fails: Log error "Failed to register table 'customeraggregatespend' in Glue Data Catalog" with exception details and raise exception to fail job
- Log info message: "Customer aggregate spend calculation complete. Total customers: [N], Total date periods: [M], Total records: [P]"
- Runtime Parameters:
- `--TARGET_AGGREGATE_PATH`: S3 path for customer aggregate spend Parquet files (default: `s3://adif-sdlc/analytics/customeraggregatespend/`)
- `--CATALOG_DATABASE`: Glue Data Catalog database name (default: `gen_ai_poc_databrickscoe`)

- `--CATALOG_AGGREGATE_TABLE``: Glue Data Catalog table name for customer aggregate spend (default: `customeraggregatespend``)

- Operational Notes:

- Aggregation is performed on active records only to reflect current state
- If historical spend analysis is required, filter by date range instead of `IsActive`` flag
- Consider partitioning by Date if query patterns support it (not specified in FRD)
- Parquet format with Snappy compression provides efficient storage for analytics
- Table is immediately queryable via Athena after successful registration
- Monitor aggregation performance for large datasets and consider optimization strategies (e.g., bucketing, caching)

-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: Python 3.7+
- Framework: PySpark 3.1+ with AWS Glue libraries
- Hudi Integration: Apache Hudi 0.10.0+ via Spark DataSource API

Execution Strategy

- Job Type: AWS Glue ETL Job (Python Shell or Spark)
- Glue Version: 3.0 or higher
- Worker Type: G.1X or G.2X (depending on data volume)
- Number of Workers: Start with 2-5 workers, scale based on data volume and performance requirements
- Job Timeout: 2880 minutes (48 hours) default, adjust based on data volume
- Max Retries: 0 (fail fast for data quality issues)
- Max Concurrent Runs: 1 (to prevent concurrent SCD2 updates)

Glue Job Type Expectations

- Recommended Job Type: Glue Spark Job (not Python Shell) due to Hudi and large-scale data processing requirements
- Spark Configuration:
 - ``spark.serializer = org.apache.spark.serializer.KryoSerializer``
 - ``spark.sql.hive.convertMetastoreParquet = false`` (for Hudi compatibility)
 - ``spark.sql.catalog.spark_catalog = org.apache.spark.sql.hudi.catalog.HoodieCatalog``
- Glue Job Parameters:
 - ``--enable-glue-datacatalog = true``

- `--enable-metrics = true``
- `--enable-continuous-cloudwatch-log = true``
- `--enable-spark-ui = true``
- `--spark-event-logs-path = s3:///spark-logs/``
- `--datalake-formats = hudi``
- `--conf spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension``
-

4. Data Design Details

Source Paths / Source Systems

Source Name	Source Type	Source Path	Format	Schema Reference
-----	-----	-----	-----	-----
Customer CSV	S3 CSV Files	<code>s3://adif-sdlc/sdlc_wizard/customerdata/`</code>	CSV with header	TR-INGEST-001 Input Schema
Order CSV	S3 CSV Files	<code>s3://adif-sdlc/sdlc_wizard/orderdata/`</code>	CSV with header	TR-INGEST-002 Input Schema
Order Summary Hudi Table	Hudi Table	<code>s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`</code>	Hudi CoW	TR-SCD2-001 Output Schema

Target Paths / Target Tables

Target Type	Target Path	Format	Catalog Database	Catalog Table	Schema Reference
-----	-----	-----	-----	-----	-----
Parquet Files	<code>s3://adif-sdlc/curated/sdlc_wizard/customer/`</code>	Parquet (Snappy)	gen_ai_poc_databrickscoe	sdlc_wizard_customer	TR-INGEST-001
Parquet Files	<code>s3://adif-sdlc/curated/sdlc_wizard/order/`</code>	Parquet (Snappy)	gen_ai_poc_databrickscoe	sdlc_wizard_order	TR-INGEST-002
Hudi Table	<code>s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`</code>	Hudi CoW	gen_ai_poc_databrickscoe	ordersummary	TR-SCD2-001
Parquet Files	<code>s3://adif-sdlc/analytics/customeraggregatespend/`</code>	Parquet (Snappy)	gen_ai_poc_databrickscoe	customeraggregatespend	TR-ANALYTICS-001

Catalog Registration Expectations

- All target tables must be registered in AWS Glue Data Catalog database `gen_ai_poc_databrickscoe``
- Catalog registration must occur atomically with data write operations
- Table metadata must accurately reflect schema, format, and location
- Hudi tables must use Hive sync for automatic catalog updates
- Parquet tables must use Glue's native catalog integration
- Catalog updates must use `UPDATE_IN_DATABASE`` behavior to handle schema evolution

Partitioning Expectations

- Customer Table: No partitioning specified in FRD (consider partitioning by Region if query patterns support it)
- Order Table: No partitioning specified in FRD (consider partitioning by Date if query patterns support it)

- Order Summary Hudi Table: No partitioning specified in FRD (Hudi supports partitioning but not required for initial implementation)
- Customer Aggregate Spend Table: No partitioning specified in FRD (consider partitioning by Date if query patterns support it)

File Formats / Table Formats

- Source Files: CSV with comma delimiter and header row
- Curated Tables: Parquet with Snappy compression
- SCD2 Table: Apache Hudi Copy-on-Write (CoW) format
- Analytics Tables: Parquet with Snappy compression

Update / Overwrite / Append Behavior

Table	Write Mode	Behavior	Rationale
-----	-----	-----	-----
izard_customer	Overwrite	Replace all data on each run	Full refresh of customer master data
izard_order	Overwrite	Replace all data on each run	Full refresh of order transaction data
ummary	Upsert (Hudi)	Insert new records, update existing records based on SCD2 logic	Maintain historical versions with SCD Type
eraggregatespend	Overwrite	Replace all data on each run	Recalculate aggregates from current active

- Note: For incremental processing (TR-INCR-001), the `ordersummary` table uses Hudi upsert to selectively update only affected records.

•

5. Processing Details

Data Quality Rules

1. Null Value Removal:
 - Remove records where any field contains the string "Null" (case-sensitive)
 - Remove records where any field is actual NULL
 - Apply to both customer and order datasets before further processing
2. Deduplication:
 - Remove exact duplicate records based on all fields
 - Retain one instance of each unique record
 - Apply to both customer and order datasets after null removal
3. Schema Validation:
 - Validate that source CSV files contain expected column names
 - Validate that data types can be inferred correctly by Spark
 - Fail job if schema validation fails

4. Data Type Validation:

- Validate that PricePerUnit can be cast to Decimal
- Validate that Qty can be cast to Integer
- Validate that Date can be parsed to Date type
- Log warnings for records that fail type validation but do not fail job (handled by null removal)

Deduplication Rules

- Deduplication Scope: All fields in the record
- Deduplication Method: Spark `dropDuplicates()` without specifying columns
- Retention Logic: Retain first occurrence (Spark default behavior)
- Execution Order: After null removal, before catalog registration

Join Rules

1. Customer-Order Join (TR-SCD2-001):

- Join Type: Inner join
- Join Key: `CustId`
- Join Logic: `customer\_df.join(order\_df, "CustId", "inner")`
- Result: All customer and order fields for matching CustId values
- Handling Unmatched Records: Unmatched records are excluded (inner join behavior)

Aggregation Rules

1. Customer Aggregate Spend (TR-AGG-001):

- Aggregation Function: SUM
- Aggregation Expression: `SUM(PricePerUnit Qty)`
- Group By Fields: `Name`, `Date`
- Filter Condition: `IsActive = True` (only active records)
- Output Field: `TotalAmount`

SCD / History Tracking Rules

1. SCD Type 2 Implementation:

- Business Key: `CustId` + `OrderId`
- Change Detection Fields: Name, EmailId, Region (customer attributes), ItemName, PricePerUnit, Qty, Date (order attributes)
- SCD2 Fields:
 - `IsActive` (Boolean): True for current active record, False for historical records
 - `StartDate` (Timestamp): Timestamp when record became active
 - `EndDate` (Timestamp): Timestamp when record became inactive (NULL for active records)

- `OpTs` (Long): Operation timestamp in milliseconds since epoch (used as Hudi precombine field)

2. SCD2 Logic:

- New Record: Insert with `IsActive = True`, `StartDate = current\_timestamp()`, `EndDate = NULL`, `OpTs = current\_timestamp\_millis()`
- Changed Record:
 - Update existing active record: Set `IsActive = False`, `EndDate = current\_timestamp()`, `OpTs = current\_timestamp\_millis()`
 - Insert new active record: Set `IsActive = True`, `StartDate = current\_timestamp()`, `EndDate = NULL`, `OpTs = current\_timestamp\_millis()`
- Unchanged Record: No action required

3. SCD2 Integrity Constraints:

- Only one active record (`IsActive = True`) per business key
- Historical records must have `IsActive = False` and non-null `EndDate`
- `StartDate` must be $\leq$ `EndDate` for historical records
- `OpTs` must be unique and monotonically increasing for each version of a record

Null Handling

- String "Null": Remove records containing the string "Null" (case-sensitive) in any field
- Actual NULL: Remove records containing actual NULL values in any field
- Post-Cleaning Expectation: All fields in cleaned datasets must be non-null
- SCD2 EndDate Exception: `EndDate` is allowed to be NULL for active records

Type Casting Expectations

- CustId: String (no casting required)
- Name: String (no casting required)
- EmailId: String (no casting required)
- Region: String (no casting required)
- OrderId: String (no casting required)
- ItemName: String (no casting required)
- PricePerUnit: Cast to Decimal during CSV parsing (Spark infers type)
- Qty: Cast to Integer during CSV parsing (Spark infers type)
- Date: Cast to String initially, can be cast to Date type for date-based operations if needed
- IsActive: Boolean (derived field)
- StartDate: Timestamp (derived field using `current\_timestamp()`)
- EndDate: Timestamp (derived field using `current\_timestamp()` or NULL)
- OpTs: Long (derived field using `current\_timestamp()` converted to milliseconds)

- TotalAmount: Decimal (calculated as PricePerUnit Qty)

-

6. Traceability Matrix

	Technical Source

	`s3://adif-sdlc/sdlc_wizard/customerdata/`
	`s3://adif-sdlc/sdlc_wizard/orderdata/`
	In-memory customer data from TR-INGEST-001
	In-memory order data from TR-INGEST-002
	In-memory customer data from TR-CLEAN-001
	In-memory order data from TR-CLEAN-002
	In-memory customer data from TR-CLEAN-003
	In-memory order data from TR-CLEAN-004
TR-CLEAN-004, Existing ordersummary Hudi table	In-memory customer data from TR-CLEAN-003, In-memory order data from TR-CLEAN-004
table	In-memory customer data from TR-CLEAN-003, `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`
	`s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

-

7. ETL Mapping Requirements

TR-INGEST-001: Customer CSV to In-Memory DynamicFrame

- Source Columns:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String (inferred)	N/A	Yes	
	Name	String (inferred)	N/A	Yes	
	EmailId	String (inferred)	N/A	Yes	
	Region	String (inferred)	N/A	Yes	

- Target Columns:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	N/A	Yes	Direct map from source CustId	
	Name	String	N/A	Yes	Direct map from source Name	
	EmailId	String	N/A	Yes	Direct map from source EmailId	
	Region	String	N/A	Yes	Direct map from source Region	

•

TR-INGEST-002: Order CSV to In-Memory DynamicFrame

• Source Columns:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String (inferred)	N/A	Yes	
	ItemName	String (inferred)	N/A	Yes	
	PricePerUnit	Decimal (inferred)	N/A	Yes	
	Qty	Integer (inferred)	N/A	Yes	
	Date	String (inferred)	N/A	Yes	
	CustId	String (inferred)	N/A	Yes	

• Target Columns:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	String	N/A	Yes	Direct map from source OrderId	
	ItemName	String	N/A	Yes	Direct map from source ItemName	
	PricePerUnit	Decimal	N/A	Yes	Direct map from source PricePerUnit	
	Qty	Integer	N/A	Yes	Direct map from source Qty	
	Date	String	N/A	Yes	Direct map from source Date	
	CustId	String	N/A	Yes	Direct map from source CustId	

•

TR-CLEAN-001: Customer Null Removal

• Source Columns:

Same as TR-INGEST-001 target columns

• Target Columns:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	N/A	No	Direct map from source CustId (filtered for non-null and not "Null")	
	Name	String	N/A	No	Direct map from source Name (filtered for non-null and not "Null")	
	EmailId	String	N/A	No	Direct map from source EmailId (filtered for non-null and not "Null")	
	Region	String	N/A	No	Direct map from source Region (filtered for non-null and not "Null")	

•

TR-CLEAN-002: Order Null Removal

- Source Columns:

Same as TR-INGEST-002 target columns

- Target Columns:

Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----	-----
OrderId	String	N/A	No	Direct map from source OrderId (filtered for non-null and not "Null")
ItemName	String	N/A	No	Direct map from source ItemName (filtered for non-null and not "Null")
PricePerUnit	Decimal	N/A	No	Direct map from source PricePerUnit (filtered for non-null and not "Null")
Qty	Integer	N/A	No	Direct map from source Qty (filtered for non-null and not "Null")
Date	String	N/A	No	Direct map from source Date (filtered for non-null and not "Null")
CustId	String	N/A	No	Direct map from source CustId (filtered for non-null and not "Null")

-

TR-CLEAN-003: Customer Deduplication

- Source Columns:

Same as TR-CLEAN-001 target columns

- Target Columns:

Same as source columns (deduplicated, retaining first occurrence)

-

TR-CLEAN-004: Order Deduplication

- Source Columns:

Same as TR-CLEAN-002 target columns

- Target Columns:

Same as source columns (deduplicated, retaining first occurrence)

-

TR-CATALOG-001: Customer to Parquet and Glue Catalog

- Source Columns:

Same as TR-CLEAN-003 target columns

- Target Columns:

Same as source columns (persisted in Parquet format)

•

TR-CATALOG-002: Order to Parquet and Glue Catalog

- Source Columns:

Same as TR-CLEAN-004 target columns

- Target Columns:

Same as source columns (persisted in Parquet format)

•

TR-SCD2-001: Customer-Order Join to Order Summary Hudi Table

- Source Columns (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	

- Source Columns (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	
	CustId	String	N/A	No	

- Target Columns:

Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----	-----
Id	String	N/A	No	Direct map from Customer.CustId (join key)
ne	String	N/A	No	Direct map from Customer.Name
ailId	String	N/A	No	Direct map from Customer.EmailId
ion	String	N/A	No	Direct map from Customer.Region
erId	String	N/A	No	Direct map from Order.OrderId
nName	String	N/A	No	Direct map from Order.ItemName

PricePerUnit	Decimal	N/A	No	Direct map from Order.PricePerUnit
	Integer	N/A	No	Direct map from Order.Qty
Date	String	N/A	No	Direct map from Order.Date
IsActive	Boolean	N/A	No	Derived: True for active records, False for historical records
StartDate	Timestamp	N/A	No	Derived: current_timestamp() when record becomes active
EndDate	Timestamp	N/A	Yes	Derived: current_timestamp() when record becomes inactive, NULL for active records
OpTs	Long	N/A	No	Derived: current_timestamp() in milliseconds since epoch

•

TR-INCR-001: Incremental Customer Updates to Order Summary

- Source Columns (Customer):

Same as TR-CLEAN-003 target columns

- Source Columns (Existing Order Summary):

Same as TR-SCD2-001 target columns

- Target Columns:

Same as TR-SCD2-001 target columns (updated records only)

- Mapping:
- Customer attributes (Name, EmailId, Region) are updated in affected order summary records
- Order attributes (OrderId, ItemName, PricePerUnit, Qty, Date) remain unchanged
- SCD2 fields (IsActive, StartDate, EndDate, OpTs) are updated according to SCD Type 2 logic

•

TR-AGG-001: Order Summary to Customer Aggregate Spend

- Source Columns:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	N/A	No	
	Name	String	N/A	No	
	EmailId	String	N/A	No	
	Region	String	N/A	No	
	OrderId	String	N/A	No	
	ItemName	String	N/A	No	
	PricePerUnit	Decimal	N/A	No	
	Qty	Integer	N/A	No	
	Date	String	N/A	No	

	IsActive	Boolean	N/A	No	
	StartDate	Timestamp	N/A	No	
	EndDate	Timestamp	N/A	Yes	
	OpTs	Long	N/A	No	

• Target Columns:

Source Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
SourceID	String	10	No	Direct map from source ID (grouped)
SourceName	String	N/A	No	Direct map from source Name (grouped)
SourceDate	String	N/A	No	Direct map from source Date (grouped)
SourceAmount	Decimal	N/A	No	Derived: SUM(PricePerUnit * Qty) grouped by Name and Date, filtered by IsActive = 1

•

• End of Technical Requirements Document